

Prompt:

8/17/2026, 9:57:44 PM

Ilanjut Tahap M

Response:

8/17/2026, 9:57:44 PM

Remembering

Continued Tahap M design by connecting forecasting inputs and clarifying QUBO transition

Worked for 7s

Tahap M — Forecasting & Factor Engine

Kita sekarang masuk ke salah satu bagian **paling penting secara matematis** dari sistem kita.

Pada Tahap L kita mengubah data mentah menjadi market state:

$$S_t = [X_t, B_t, \Sigma_{F,t}, \Sigma_{\epsilon,t}, regime_t, liquidity_t, w_{t-1}].$$

Tahap M menjawab pertanyaan:

Dari state pasar S_t , berapa expected return setiap aset pada horizon trading berikutnya, dan bagaimana ketidakpastian serta hubungan antar-aset berubah?

Output utama kita adalah:

$$\hat{\mu}_t$$

dan

$$\hat{\Sigma}_t.$$

Keduanya nantinya menjadi bahan baku langsung untuk optimasi portfolio/QUBO.

M.1. Arsitektur Tahap M

Pipeline kita:

$$S_t \rightarrow \text{Factor Forecast} \rightarrow \text{Asset Forecast} \rightarrow \text{Residual Forecast} \rightarrow \hat{\mu}_t$$

dan secara paralel:

$$S_t \rightarrow B_t \rightarrow \Sigma_{F,t} \rightarrow \Sigma_t$$

sehingga:

$$(\hat{\mu}_t, \Sigma_t) \rightarrow \text{Portfolio Optimization.}$$

M.2. Jangan langsung memprediksi harga

Ini keputusan desain penting.

Kita **tidak** ingin:

$$\hat{P}_{t+1} = f(P_t).$$

Lebih masuk akal menggunakan return:

$$r_{i,t+1} = \ln \frac{P_{i,t+1}}{P_{i,t}}.$$

Kemudian:

$$\mu_{i,t} = E[r_{i,t+1} \mid \mathcal{F}_t].$$

Alasannya sederhana:

harga bersifat non-stationary, sedangkan return relatif lebih cocok untuk model statistik.

M.3. Horizon trading

Karena sistem kita ditujukan untuk short-term trading, kita perlu menentukan:

$$\Delta.$$

Untuk V1 kita gunakan:

$$\Delta = 1 \text{ jam.}$$

Maka:

$$r_{i,t+1} = \ln \frac{P_{i,t+1h}}{P_{i,t}}.$$

Kemudian:

$$\mu_{i,t} = E[r_{i,t+1h} \mid F_t].$$

M.4. Mengapa expected return saja tidak cukup?

Misalkan:

$$\mu_A = 0.20\%$$

dan:

$$\mu_B = 0.15\%.$$

Secara naif kita memilih A.

Tetapi ternyata:

$$\sigma_A = 1.5\%$$

sedangkan:

$$\sigma_B = 0.3\%.$$

Maka A memiliki return lebih tinggi tetapi risk yang jauh lebih besar.

Karena itu kita membutuhkan:

$$(\mu, \Sigma)$$

bukan hanya:

$$\mu.$$

M.5. Model factor dasar

Misalkan terdapat N asset dan K factor.

Kita definisikan:

$$\mathbf{r}_t = \boldsymbol{\alpha}_t + B_t \mathbf{f}_t + \boldsymbol{\epsilon}_t.$$

Dengan:

$$\mathbf{r}_t \in \mathbb{R}^N$$

dan:

$$\mathbf{f}_t \in \mathbb{R}^K.$$

M.6. Contoh factor kita

Misalnya:

$$\mathbf{f}_t = \begin{bmatrix} r_{BTC,t} \\ r_{ETH,t} \\ r_{DXY,t} \\ r_{Gold,t} \\ r_{SPX,t} \\ r_{VIX,t} \end{bmatrix}.$$

Untuk saham internasional nanti factor tersebut dapat diperluas.

Tetapi prinsipnya tetap sama.

M.7. Matrix exposure

Untuk asset i :

$$\beta_i = \begin{bmatrix} \beta_{i,BTC} \\ \beta_{i,ETH} \\ \beta_{i,DXY} \\ \beta_{i,Gold} \\ \beta_{i,SPX} \\ \beta_{i,VIX} \end{bmatrix}.$$

Seluruh asset:

$$B = \begin{bmatrix} \beta_1^T \\ \beta_2^T \\ \vdots \\ \beta_N^T \end{bmatrix}.$$

Sehingga:

$$B \in \mathbb{R}^{N \times K}.$$

M.8. Forecast factor

Sekarang kita tidak hanya ingin mengetahui:

$$f_t.$$

Kita ingin:

$$\hat{\mathbf{f}}_{t+1} = E[\mathbf{f}_{t+1} \mid \mathbf{F}_t].$$

Ini sangat penting.

Misalnya sistem mendeteksi:

$$\hat{r}_{BTC,t+1} > 0,$$

tetapi:

$$\hat{r}_{DXY,t+1} > 0.$$

Keduanya dapat memberikan pengaruh berbeda terhadap asset yang sama.

M.9. Forecast asset dari factor

Dari:

$$r_{i,t+1} = \alpha_i + \beta_i^T f_{t+1} + \epsilon_{i,t+1},$$

ambil conditional expectation:

$$E[r_{i,t+1} | F_t].$$

Maka:

$$\mu_{i,t} = \alpha_i + \beta_i^T E[f_{t+1} | F_t] + E[\epsilon_{i,t+1} | F_t].$$

Sehingga:

$$\hat{\mu}_{i,t} = \alpha_{i,t} + \hat{\beta}_{i,t}^T \hat{f}_{t+1} + \hat{\epsilon}_{i,t+1}.$$

Ini adalah persamaan inti Factor Engine kita.

M.10. Kita pecah expected return menjadi tiga bagian

$$\hat{\mu}_i = \underbrace{\alpha_i}_{\text{baseline}} + \underbrace{\hat{\beta}_i^T \hat{f}}_{\text{factor contribution}} + \underbrace{\hat{\epsilon}_i}_{\text{idiosyncratic contribution}}.$$

Interpretasinya:

1. Baseline

$$\alpha_i.$$

2. Market/factor-driven component

$$\hat{\beta}_i^T \hat{f}.$$

3. Asset-specific component

$$\hat{\epsilon}_i.$$

M.11. Mengapa residual prediction penting?

Bayangkan:

$$BTC \uparrow .$$

Sebuah altcoin memiliki:

$$\beta_{BTC} = 1.5.$$

Secara factor model:

$$r_{alt} \approx 1.5 r_{BTC}.$$

Tetapi ternyata asset tersebut sedang mengalami:

$$\epsilon_t < 0.$$

Maka forecast sebenarnya:

$$\hat{\mu} = 1.5 r_{BTC} + \hat{\epsilon}.$$

Ini jauh lebih informatif daripada sekadar:

BTC naik → beli altcoin.

M.12. Residual bukan noise yang harus dibuang

Pada model sederhana:

$$r = Bf + \epsilon,$$

orang sering menganggap:

$$\epsilon$$

sebagai noise.

Tetapi untuk trading:

$$\boxed{\epsilon_t}$$

dapat mengandung alpha.

Misalnya residual mengalami mean reversion:

$$\epsilon_{t+1} = \phi \epsilon_t + \eta_{t+1}.$$

Jika:

$$-1 < \phi < 0,$$

residual memiliki kecenderungan reversal.

M.13. Model residual AR(1)

Kita dapat menggunakan:

$$\epsilon_{i,t+1} = c_i + \phi_i \epsilon_{i,t} + \eta_{i,t+1}.$$

Maka:

$$E[\epsilon_{i,t+1} | F_t] = c_i + \phi_i \epsilon_{i,t}.$$

Jadi:

$$\hat{\epsilon}_{i,t+1} = \hat{c}_i + \hat{\phi}_i \epsilon_{i,t}.$$

M.14. Jika $\phi < 0$

Misalkan:

$$\phi = -0.4$$

dan:

$$\epsilon_t = -1\%.$$

Maka:

$$\hat{\epsilon}_{t+1} = (-0.4)(-1\%) = +0.4\%.$$

Artinya asset sedang mengalami underperformance terhadap factor model dan model memperkirakan sebagian reversal.

M.15. Jika $\phi > 0$

Misalkan:

$$\phi = 0.4.$$

Jika:

$$\epsilon_t = 1\%,$$

maka:

$$\hat{\epsilon}_{t+1} = 0.4\%.$$

Ini lebih menyerupai momentum.

Jadi:

$$\boxed{\phi}$$

membantu menentukan karakteristik residual.

M.16. Tetapi jangan langsung percaya residual

Kita harus menguji apakah:

$$\hat{\phi}$$

stabil.

Rolling estimation:

$$\phi_t = \text{OLS}(\epsilon_{t-W:t}).$$

Jika:

$$\phi_t$$

berubah-ubah drastis, residual model tidak stabil.

Maka confidence terhadap signal harus turun.

M.17. Confidence-adjusted expected return

Ini ide penting untuk sistem kita.

Jangan hanya menggunakan:

$$\hat{\mu}.$$

Tambahkan:

$$q_i \in [0, 1]$$

sebagai confidence.

Kemudian:

$$\tilde{\mu}_i = q_i \hat{\mu}_i.$$

Jika model sangat yakin:

$$q_i \approx 1.$$

Jika tidak yakin:

$$q_i \approx 0.$$

M.18. Apa yang menentukan confidence?

Kita dapat membangun:

$$q_i = f(R^2, \text{parameter stability, sample size, data quality, regime stability}).$$

Misalnya:

$$q_i = q_{data} q_{model} q_{regime}.$$

M.19. Jangan membuat confidence secara arbitrer

Untuk V1, lebih baik kita membuatnya sebagai fungsi yang sederhana dan dapat diuji.

Misalnya:

$$q_i = \text{clip}\left(\frac{IC_i}{IC_{max}}, 0, 1\right).$$

dengan Information Coefficient:

$$IC = \text{Corr}(\mu_t, r_{t+1}).$$

M.20. Information coefficient

Jika:

$$IC > 0,$$

forecast cenderung benar secara arah.

Jika:

$$IC \approx 0,$$

forecast tidak berguna.

Jika:

$$IC < 0,$$

model bahkan berlawanan arah.

Namun:

$$IC$$

harus dihitung **out-of-sample**.

M.21. Forecast factor dengan model sederhana terlebih dahulu

Untuk V1 kita tidak membutuhkan deep learning.

Kita dapat mulai dengan:

$$\hat{f}_{t+1} = A_t f_t + c_t.$$

Ini adalah VAR(1).

M.22. VAR(1)

Dengan:

$$\mathbf{f}_{t+1} = \mathbf{c} + A\mathbf{f}_t + \boldsymbol{\eta}_{t+1}.$$

Maka:

$$\hat{\mathbf{f}}_{t+1} = \mathbf{c} + A\mathbf{f}_t.$$

M.23. Mengapa VAR menarik?

Karena factor kita saling berhubungan.

Misalnya:

$$BTC \leftrightarrow ETH$$

dan:

$$DXY \leftrightarrow Gold \leftrightarrow SPX.$$

VAR menangkap cross-factor dynamics:

$$A_{ij} \not\equiv 0.$$

M.24. Contoh

Misalnya:

$$\hat{r}_{BTC,t+1} = 0.2r_{BTC,t} - 0.1r_{DXY,t}$$

dan:

$$\hat{r}_{ETH,t+1} = 0.5r_{BTC,t} + 0.1r_{ETH,t}.$$

Maka sistem tidak hanya bertanya:

bagaimana BTC bergerak?

tetapi:

bagaimana kondisi seluruh factor memengaruhi perkembangan factor berikutnya?

M.25. Namun VAR memiliki masalah

Jika:

$$K = 6$$

dan kita menggunakan lag:

$$p = 10,$$

jumlah parameter cepat meningkat.

Dengan:

$$K^2p$$

parameter.

Karena itu kita dapat menggunakan ridge:

$$\hat{A} = \arg \min_A \{ \|F_{future} - AF_{past}\|_F^2 + \lambda \|A\|_F^2 \}.$$

M.26. Forecast factor uncertainty

Forecast tidak hanya:

$$\hat{f}.$$

Kita juga membutuhkan:

$$\Sigma_{\hat{f}}$$

Misalkan:

$$f_{t+1} = c + Af_t + \eta_{t+1}$$

dengan:

$$\text{Cov}(\eta) = \Omega.$$

Untuk one-step forecast:

$$\Sigma_{\hat{f}} = \Omega.$$

M.27. Asset covariance

Kembali ke:

$$r_t = B_t f_t + \epsilon_t.$$

Jika:

$$\text{Cov}(f_t) = \Sigma_F$$

dan:

$$\text{Cov}(\epsilon_t) = \Sigma_\epsilon,$$

serta factor dan residual dianggap tidak berkorelasi:

$$\text{Cov}(f, \epsilon) = 0,$$

maka:

$$\Sigma_r = B \Sigma_F B^T + \Sigma_\epsilon.$$

Ini adalah persamaan inti risk engine.

M.28. Penurunan matematis

Mulai:

$$r = Bf + \epsilon.$$

Covariance:

$$\text{Cov}(r) = E[(r - E[r])(r - E[r])^T].$$

Karena:

$$r - E[r] = B(f - E[f]) + (\epsilon - E[\epsilon]),$$

maka:

$$\text{Cov}(r) = B \Sigma_F B^T + \Sigma_\epsilon + B \text{Cov}(f, \epsilon) + \text{Cov}(\epsilon, f) B^T.$$

Jika:

$$\text{Cov}(f, \epsilon) = 0,$$

maka:

$$\Sigma_r = B \Sigma_F B^T + \Sigma_\epsilon.$$

M.29. Mengapa ini lebih baik daripada covariance langsung?

Covariance sample biasa:

$$\hat{\Sigma} = \frac{1}{T-1} RR^T.$$

Masalahnya:

- noisy,
- unstable,
- sensitive terhadap regime,
- buruk jika N besar dan T relatif kecil.

Factor decomposition memberikan struktur:

$$\boxed{\text{Systematic Risk} + \text{Idiosyncratic Risk.}}$$

M.30. Factor risk

$$\boxed{\Sigma_{factor} = B \Sigma_F B^T.}$$

Jika dua asset mempunyai:

$$\beta_{i,BTC} > 0$$

dan:

$$\beta_{j,BTC} > 0,$$

mereka dapat memiliki covariance tinggi meskipun tidak selalu terlihat dari short sample.

M.31. Idiosyncratic risk

$$\boxed{\Sigma_{\epsilon^*}}$$

Pada model paling sederhana:

$$\Sigma_{\epsilon} = \text{diag}(\sigma_{\epsilon,1}^2, \dots, \sigma_{\epsilon,N}^2).$$

Tetapi ini terlalu sederhana.

M.32. Residual correlation

Kita dapat mengestimasi:

$$\text{Corr}(\epsilon_i, \epsilon_j).$$

Jika signifikan, maka:

$$\Sigma_{\epsilon}$$

tidak diagonal.

Ini memungkinkan kita menemukan hubungan yang tidak dijelaskan factor global.

M.33. Shrinkage

Sample covariance residual dapat noisy.

Gunakan:

$$\Sigma_{\epsilon}^{\text{shrunk}} = (1 - \lambda)\hat{\Sigma}_{\epsilon} + \lambda T.$$

Misalnya:

$$T = \text{diag}(\hat{\Sigma}_{\epsilon}).$$

M.34. Dynamic covariance

Pasar short-term berubah.

Maka:

$$\Sigma_t \neq \Sigma_{t-1}.$$

Kita membutuhkan dynamic estimation.

Salah satu baseline yang sangat baik:

$$\Sigma_t^{\text{EWMA}} = \lambda \Sigma_{t-1}^{\text{EWMA}} + (1 - \lambda) r_{t-1} r_{t-1}^T.$$

M.35. Factor covariance EWMA

Kita dapat menerapkan hal yang sama pada factor:

$$\Sigma_{F,t} = \lambda_F \Sigma_{F,t-1} + (1 - \lambda_F) f_{t-1} f_{t-1}^T.$$

Dengan demikian covariance lebih responsif terhadap perubahan regime.

M.36. Volatility regime

Misalnya:

$$\sigma_t^{short}$$

dibandingkan:

$$\sigma_t^{long}.$$

Definisikan:

$$VR_t = \frac{\sigma_t^{short}}{\sigma_t^{long}}.$$

Jika:

$$VR_t \gg 1,$$

kita berada pada high-volatility regime.

M.37. Mengapa regime penting untuk μ ?

Signal momentum yang bekerja pada:

low volatility

belum tentu bekerja pada:

high volatility.

Jadi:

$$\mu_t = \mu(S_t)$$

bukan konstanta.

M.38. Regime-dependent forecast

Secara sederhana:

$$\hat{\mu}_t = \begin{cases} \hat{\mu}_t^{low}, & \text{Regime} = Low \\ \hat{\mu}_t^{high}, & \text{Regime} = High. \end{cases}$$

Tetapi kita tidak perlu langsung membuat dua model terpisah.

Kita dapat memasukkan regime sebagai feature:

$$X_t = [returns, volatility, factors, regime].$$

M.39. Mean model

Sekarang kita dapat membangun model:

$$\hat{\mu}_{i,t} = \alpha_i + \beta_i^T \hat{f}_{t+1} + \phi_i \epsilon_{i,t} + \theta_i^T X_{i,t}.$$

Ini menjadi baseline forecasting engine kita.

M.40. Tetapi hati-hati: jangan terlalu banyak parameter

Kita tidak ingin membuat:

$$\hat{\mu} = f(1000 \text{ features})$$

dengan dataset terbatas.

Karena:

Overfitting

akan menjadi musuh utama.

V1 harus **sederhana tetapi mathematically defensible**.

M.41. Maka arsitektur V1 saya tetapkan

Saya sarankan:

$$\hat{\mu} = \hat{\mu}^{factor} + \hat{\mu}^{residual} + \hat{\mu}^{momentum}$$

dengan:

$$\hat{\mu}^{factor} = B\hat{f},$$

$$\hat{\mu}^{residual} = \phi\epsilon_t,$$

dan:

$$\hat{\mu}^{momentum} = \theta^T r_t^{multi-horizon}.$$

M.42. Secara eksplisit

Untuk asset i :

$$\hat{\mu}_{i,t} = \alpha_i + \beta_i^T \hat{f}_{t+1} + \phi_i \epsilon_{i,t} + \theta_{i,1} r_{i,t}^{1h} + \theta_{i,2} r_{i,t}^{4h} + \theta_{i,3} r_{i,t}^{24h}.$$

Ini sudah cukup kaya untuk V1.

M.43. Regularisasi

Parameter:

$$\theta, \beta, \phi$$

dapat diestimasi dengan ridge:

$$\hat{w} = \arg \min_w [\|y - Xw\|_2^2 + \lambda \|w\|_2^2].$$



Solusinya:

$$\hat{w} = (X^T X + \lambda I)^{-1} X^T y.$$

M.44. Mengapa ridge cocok?

Karena features kita dapat berkorelasi.

Contoh:

$$r_{BTC}^{1h}$$

dan:

$$r_{BTC}^{4h}$$

jelas tidak independen.

Begitu pula:

$$DXY$$

dan:

$$Gold.$$

Ridge mengurangi coefficient explosion.

M.45. Standardisasi

Sebelum ridge:

$$x_j$$

harus dinormalisasi menggunakan training window.

$$z_j = \frac{x_j - \mu_j^{train}}{\sigma_j^{train}}.$$

Kemudian model dilatih.

M.46. Forecast output

Pada setiap timestamp:

t

engine menghasilkan:

$$\hat{\mu}_t = \begin{bmatrix} \hat{\mu}_{1,t} \\ \hat{\mu}_{2,t} \\ \vdots \\ \hat{\mu}_{N,t} \end{bmatrix}.$$

Contoh:

$$\hat{\mu}_t = \begin{bmatrix} 0.0018 \\ 0.0009 \\ -0.0003 \\ 0.0021 \end{bmatrix}.$$

Artinya:

0.18%, 0.09%, -0.03%, 0.21%

expected log-return satu jam ke depan.

M.47. Tetapi expected return harus dibandingkan dengan biaya

Ini sangat penting untuk trading.

Jika:

$$\hat{\mu}_i = 0.08\%$$

tetapi:

transaction cost = 0.10%,

maka:

$$\boxed{NetAlpha_i < 0.}$$

Karena itu kita definisikan:

$$\boxed{\alpha_i^{net} = \mu_i - C_i.}$$

M.48. Cost model

Minimal:

$$C_i = fee_i + spread_i + slippage_i.$$

Sehingga:

$$\boxed{\tilde{\mu}_i = \mu_i - C_i.}$$

Ini adalah expected return yang seharusnya masuk optimizer.

M.49. Market impact

Untuk order yang lebih besar:

$$Impact(q)$$

meningkat terhadap ukuran order.

Model sederhana:

$$Impact(q) = \eta \left(\frac{q}{ADV} \right)^\gamma.$$

Maka:

$$C(q) = fee + spread + Impact(q).$$

Ini nantinya menjadi bagian penting dari optimasi position sizing.

M.50. Kita sekarang mempunyai problem optimasi yang lebih realistis

Bukan:

$$\max_w (\mu^T w - \gamma w^T \Sigma w).$$

Tetapi:

$$\max_w [\tilde{\mu}^T w - \gamma w^T \Sigma w - C(w)].$$

Di sinilah sistem kita mulai berbeda dari portfolio optimization akademik sederhana.

M.51. Hubungan dengan risk budget

Karena sebelumnya kita ingin setiap trade mengikuti:

$$\$1$$

risk budget, kita nantinya akan menghubungkan:

$$w_i$$

dengan:

$$SL_i.$$

Jika stop distance:

$$d_i$$

dan position size:

$$q_i,$$

maka approximate dollar risk:

$$R_i = q_i d_i.$$

Constraint:

$$R_i \leq R_{\max}.$$

Misalnya:

$$R_{\max} = 1.$$

M.52. Expected return menjadi expected PnL

Jika:

$$q_i$$

unit asset dan:

$$\hat{\mu}_i$$

expected return:

$$E[PnL_i] \approx q_i P_i \hat{\mu}_i.$$

Sedangkan risk:

$$\sigma_{PnL,i} \approx q_i P_i \sigma_i.$$

Jadi:

$$\text{forecast layer} \rightarrow \text{position layer}$$

mulai terbentuk.

M.53. Sekarang covariance menjadi sangat penting

Untuk portfolio:

$$w = \begin{bmatrix} w_1 \\ \vdots \\ w_N \end{bmatrix},$$



expected return:

$$E[R_p] = w^T \mu.$$

Risk:

$$\text{Var}(R_p) = w^T \Sigma w.$$

M.54. Ini langsung menuju QUBO

Misalnya binary decision:

$$x_i \in \{0, 1\}.$$

Kita definisikan:

$$w_i = q_i x_i.$$

Objective:

$$\max_x \left[\sum_i \mu_i q_i x_i - \gamma \sum_{i,j} \Sigma_{ij} q_i q_j x_i x_j \right].$$

Karena:

$$x_i^2 = x_i,$$

maka objective dapat ditulis sebagai:

$$\max_x \left[\sum_i a_i x_i - \sum_{i < j} b_{ij} x_i x_j \right].$$

Ini adalah QUBO.

M.55. Koefisien linear

$$a_i = \tilde{\mu}_i q_i - \gamma \Sigma_{ii} q_i^2.$$

Jadi expected return forecast mempengaruhi:

$$Q_{ii}.$$

M.56. Koefisien interaksi

Untuk $i \exists j$:

$$b_{ij} = 2\gamma \Sigma_{ij} q_i q_j.$$

Jadi covariance forecast menentukan interaksi:

$$Q_{ij}.$$

Ini sangat penting:

Factor Engine bukan modul terpisah dari QAOA.

Outputnya secara langsung menentukan landscape energi yang akan dieksplorasi quantum optimizer.

M.57. Hubungan lengkap

Sekarang kita memiliki:

Market Data



Factors



$$\mu, \Sigma$$

↓

$QUBO$

↓

H_C

↓

$QAOA$

↓

$Portfolio/TradeSelection$

M.58. Bahkan factor exposure dapat masuk ke constraint

Misalnya kita tidak ingin portfolio terlalu exposed terhadap BTC.

Portfolio beta:

$$\beta_p = B^T w.$$

Kita dapat memberikan constraint:

$$|\beta_{p,BTC}| \leq \beta_{max}.$$

Begitu juga:

$$|\beta_{p,DXY}| \leq \beta_{DXY,max}.$$

Ini memungkinkan kita mengontrol **factor risk**, bukan hanya volatility.

M.59. Factor-neutral portfolio

Jika kita menginginkan:

$$\beta_p \approx 0,$$

maka:

$$\boxed{B^T w \approx 0.}$$

Ini menghasilkan portfolio yang relatif netral terhadap factor tertentu.

Contohnya:

$$\beta_{p,DXY} \approx 0.$$

Dengan demikian profit/loss lebih banyak berasal dari:

idiosyncratic alpha.

M.60. Tetapi untuk sistem trading kita

Kita tidak selalu ingin factor-neutral.

Karena jika model memperkirakan:

$$\hat{r}_{BTC} > 0,$$

kita mungkin **memang ingin mengambil BTC exposure**.

Jadi factor exposure sebaiknya menjadi:

$$\boxed{\text{decision variable}}$$

bukan selalu constraint nol.

M.61. Dynamic factor exposure

Kita dapat menentukan target:

$$\beta_p^*$$

berdasarkan regime.

Misalnya:

$$\beta_{BTC,p}^* = f(r_{BTC}, \sigma_{BTC}, regime).$$

Kemudian optimizer mencari portfolio yang mendekati target tersebut.

Ini sudah jauh lebih advanced.

M.62. Tetapi belum kita masukkan ke V1

Untuk menjaga eksperimen tetap dapat dibuktikan, V1:

$$\boxed{\text{factor exposure} \rightarrow \mu, \Sigma}$$

sedangkan:

$$factor\ constraints$$

kita masukkan setelah baseline tervalidasi.

Ini penting untuk menghindari sistem menjadi terlalu kompleks sebelum kita tahu apakah alpha dasarnya benar-benar ada.

M.63. Output final Tahap M

Pada setiap waktu t , Factor Engine harus menghasilkan object konseptual:

$$\boxed{M_t = \{\hat{\mu}_t, \Sigma_t, B_t, \hat{f}_{t+1}, \Sigma_{F,t}, \Sigma_{\epsilon,t}, \hat{\epsilon}_t, C_t, Q_t\}}.$$

Dengan:

Forecast

$$\hat{\mu}_t.$$

Risk

$$\Sigma_t.$$

Factor exposure

$$B_t.$$

Factor forecast

$$\hat{f}_{t+1}.$$

Residual forecast

$$\hat{\epsilon}_t.$$

Trading cost

$$C_t.$$

Confidence-adjusted alpha

$$Q_t.$$

M.64. Quality control Tahap M

Sebelum kita menggunakan output ini untuk trading, kita harus menguji:

1. Directional accuracy

$$P(\text{sign}(\hat{\mu}_t) = \text{sign}(r_{t+1})).$$

2. Information coefficient

$$IC = \text{Corr}(\hat{\mu}_t, r_{t+1}).$$

3. Calibration

Apakah:

$$\hat{\mu} = 0.2\%$$

benar-benar menghasilkan realized return sekitar:

$$0.2\%$$

secara statistik?

4. Stability

Apakah parameter stabil terhadap waktu?

5. Turnover

Apakah signal berubah terlalu cepat?

M.65. Yang paling penting: forecast tidak dinilai dari satu trade

Misalnya model mengatakan:

$$\hat{\mu} = 0.15\%.$$

Tetapi realized:

$$r = -0.3\%.$$

Itu tidak berarti model gagal.

Yang kita evaluasi adalah:

$$E[r \mid \hat{\mu}].$$

Jika secara agregat:

$$\hat{\mu} \uparrow \Rightarrow r \uparrow,$$

model memiliki predictive power.

M.66. Kita juga harus membandingkan dengan baseline

Model kita tidak boleh hanya dibandingkan dengan dirinya sendiri.

Minimal:

Baseline 1 : $\mu = 0$

Baseline 2 : historical mean

Baseline 3 : momentum only

Baseline 4 : factor model

Model 5 : factor + residual + momentum.

Baru kita dapat mengetahui apakah kompleksitas tambahan memang menghasilkan alpha.

M.67. Ini juga penting untuk tesis quantum kita

Karena nanti kita ingin membandingkan:

Classical optimizer

vs.

QAOA.

Kita **tidak boleh** membiarkan perbedaan forecasting membuat perbandingan optimizer menjadi tidak fair.

Maka:

same μ_t and same Σ_t

harus diberikan kepada:

Classical, Exact, QAOA.

Dengan demikian yang dibandingkan benar-benar:

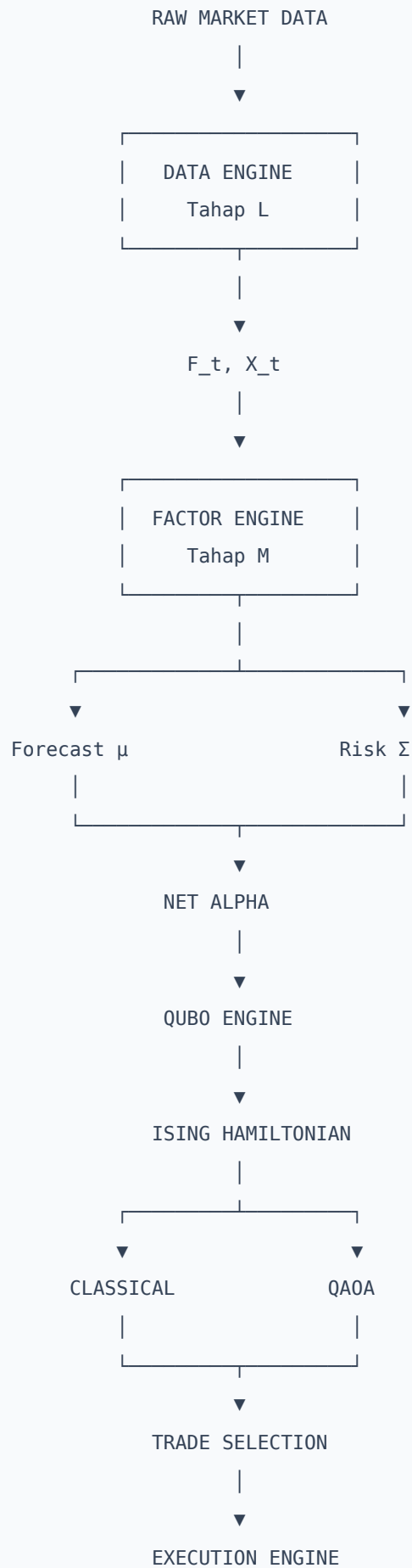
Optimization method

bukan:

Data/forecasting method.

M.68. Arsitektur lengkap sampai Tahap M

Sekarang sistem kita sudah menjadi:



M.69. Satu hal yang sangat penting

Sekarang kita dapat melihat bahwa **QAOA bukan predictor harga**.

QAOA tidak perlu mengetahui:

$$BTC = 115000.$$

Ia juga tidak perlu mengetahui:

$$DXY = 98.3.$$

Semua informasi tersebut sudah diproses menjadi:

$$\hat{\mu} \text{ dan } \Sigma.$$

QAOA hanya menyelesaikan:

$$\text{given current expected returns and risk structure,}$$

kombinasi trade mana yang secara matematis paling optimal?

Ini adalah pemisahan yang sangat sehat antara:

$$Prediction \boxminus Optimization.$$

M.70. Dan inilah inti sistem kita

Secara keseluruhan:

$$F_t \rightarrow S_t \rightarrow (\hat{\mu}_t, \Sigma_t) \rightarrow Q_t \rightarrow H_C \rightarrow QAOA \rightarrow x_t^*$$

dengan:

$$\hat{\mu}_t = E[r_{t+1} | F_t]$$

dan:

$$\Sigma_t = B_t \Sigma_{F,t} B_t^T + \Sigma_{\epsilon,t}.$$

Kemudian:

$$Q_t = Q(\hat{\mu}_t, \Sigma_t, C_t, R_t)$$

dan akhirnya:

$$x_t^* = \arg \min_{x \in \{0,1\}^n} x^T Q_t x.$$

Itulah jembatan matematis yang kita cari sejak awal.

Apa yang berubah setelah Tahap M?

Sebelum Tahap M, kita hanya mempunyai:

market data.

Sekarang kita mempunyai **belief matematis tentang masa depan pasar**:

$$\hat{\mu}_t$$

serta **geometri risiko pasar**:

$$\Sigma_t.$$

Dan keduanya sekarang sudah siap diubah menjadi **masalah optimasi diskret**.

Tahap berikutnya yang paling natural adalah **Tahap N — Dynamic QUBO Construction & Position Sizing**.

Di sana kita bisa menurunkan secara penuh, mulai dari

$$\max_w (\mu^T w - \gamma w^T \Sigma w),$$

kemudian memasukkan **transaction cost, SL/TP, \$1 risk budget, liquidity, maximum concurrent positions, long/short, cardinality, dan rebalancing**, lalu secara sistematis mengubahnya menjadi

$$x^T Q x$$

dan akhirnya ke

$$H_C = \sum_i h_i Z_i + \sum_{i < j} J_{ij} Z_i Z_j.$$

Di titik itulah koneksi antara **quantitative trading yang realistis** dan **QAOA/Ising optimization** akan menjadi sangat konkret.